

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Imagenes/logo.uamzm.jpg

Imagenes/LOGO.png

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA

PRÁCTICA 6

Encendido de un LED mediante la
pulsación de un botón en RPi Pico W

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 20 DE MAYO DE 2026

Encendido de un LED Mediante la Pulsación de un Botón en RPi Pico W

e Implementación de Alimentación Externa

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Este reporte describe el desarrollo de la Práctica 6, enfocada en la configuración de los pines de propósito general de entrada/salida (GPIO) de la Raspberry Pi Pico W mediante la implementación de un botón pulsador como entrada digital y un LED externo como salida. El control lógico se estructuró en lenguaje C utilizando el SDK oficial de Raspberry Pi, incorporando una rutina de software (*software debounce*) para mitigar los rebotes mecánicos del interruptor. Adicionalmente, se abordó la desconexión del puerto USB de control mediante la implementación de una fuente de alimentación externa acoplada directamente a los pines correspondientes del microcontrolador, garantizando la autonomía operativa del sistema embebido.

I. INTRODUCCIÓN

I-A Objetivos de Aprendizaje

El desarrollo experimental de esta práctica se rige bajo los siguientes objetivos esenciales:

- Aprender a configurar los pines de propósito general de entrada/salida (GPIO) de la placa RPi Pico W al implementar un botón como entrada y un LED como salida en el microcontrolador.
- Aprender a conectar una fuente de alimentación al microcontrolador sin depender del puerto USB.

I-B Entradas y Salidas Digitales en Sistemas Embebidos

La lectura de señales digitales externas y la respectiva manipulación de actuadores visuales constituyen la base operativa de cualquier sistema embebido en la ingeniería mecatrónica. El microcontrolador RP2040 permite que sus líneas GPIO actúen tanto en modo de alta impedancia para lecturas digitales como en modo de drenador/fuente de corriente para activar periféricos. La lectura correcta de un interruptor mecánico requiere de estados eléctricos estables y bien definidos (alto o bajo), evitando tensiones flotantes mediante arreglos resistivos físicos de acoplamiento.

I-C El Problema de los Rebotes Mecánicos (*Bouncing*)

Al presionar un pulsador sin enclavamiento mecánico, las láminas internas no hacen contacto de forma limpia instantáneamente, sino que oscilan físicamente produciendo una serie de pulsos falsos de apertura y cierre rápidos antes de estabilizarse. En software de microcontroladores de alta velocidad, estas transiciones espurias de microsegundos pueden ser interpretadas erróneamente como múltiples pulsaciones consecutivas. Por lo tanto, el uso de técnicas de retardo controlado o temporización por software (*debouncing*) es crítico para asegurar la fidelidad de la entrada.

II. DESARROLLO

II-A Material y Recursos Utilizados

Para el correcto armado y ejecución de la práctica se emplearon los siguientes elementos de hardware y software:

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 1 LED (verde de preferencia), 1 fuente de alimentación externa (3.3V–5.5V), 1 resistencia de 45 Ω (calculada para LED verde con una corriente de 20 mA), 1 resistencia de 10 k Ω y 1 pulsador sin enclavamiento mecánico.
- **Software y librerías:** SDK de Raspberry Pi Pico, entorno embebido Pico – Visual Studio Code y la biblioteca estándar de hardware `pico_stdlib`.

II-B Estructura del Proyecto y Configuración de CMake

Se acondicionó la carpeta de proyecto `PRACTICA_6` inyectando el script `pico_sdk_import.cmake` y los subdirectorios del entorno `.vscode`. La orquestación del compilador cruzado se definió en el archivo `CMakeLists.txt` que se expone a continuación:

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_6)

pico_sdk_init()

add_executable(Pulsador
    Pulsador.c
)

target_link_libraries(Pulsador
    pico_stdlib
)

pico_add_extra_outputs(Pulsador)
```

El script vincula formalmente el código fuente al binario final `Pulsador`, inyectando la macro de inicialización del SDK y exportando el formato compilado ejecutable estructurado en la extensión `.uf2`.

II-C Implementación de la Lógica de Control en C

El comportamiento lógico del sistema se programó en el archivo `Pulsador.c`. El algoritmo inicializa el pin GPIO3 como salida digital y GPIO4 como entrada digital, para posteriormente entrar a un lazo síncrono infinito que evalúa de forma persistente la señal del botón implementando una técnica de filtrado de rebotes de 50 ms.

CÓDIGO FUENTE – Pulsador.c

```
#include "pico/stdlib.h"

int main() {
    const uint LED_PIN = 3;
    const uint BOTON = 4;

    gpio_init(LED_PIN);
    gpio_init(BOTON);
    gpio_set_dir(LED_PIN, GPIO_OUT);
    gpio_set_dir(BOTON, GPIO_IN);

    while(true) {
        if (gpio_get(BOTON)) {
            sleep_ms(50); //
            Debounce: espera 50 ms
            if (gpio_get(BOTON)) {
                gpio_put(LED_PIN, 1); //
                Enciende LED
            }
        } else {
```

```
        gpio_put(LED_PIN, 0); //
        Apaga LED
    }
    sleep_ms(10);
}
```

II-D Conexión del Circuito y Alimentación Externa

El circuito físico se estructuró sobre una protoboard siguiendo las directrices de la topología *Pull-Down*. Se acopló el pulsador a la línea de alimentación de 3.3V, referenciando el pin GPIO 4 a tierra física (GND) mediante una resistencia de 10 kΩ. El LED verde se dispuso en serie con una resistencia limitadora de 45 Ω conectada directamente al pin GPIO 3.

Para cumplir el segundo objetivo principal, se retiró el cable micro USB tras la carga del archivo `.uf2` y se integró una fuente de alimentación regulada independiente conectada a los rieles laterales de la protoboard, cuya línea positiva se conecta al pin **VSYS** (pin físico 39) y la negativa a cualquier pin **GND** de la placa.

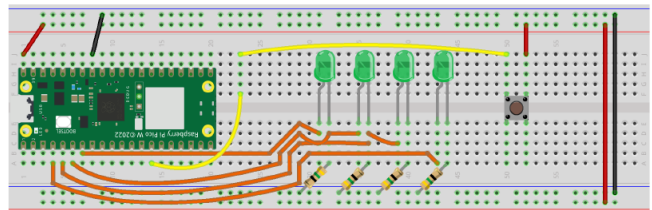


Figura 1: Diagrama de conexiones en protoboard: botón pulsador (Pull-Down), LED con resistencia limitadora y alimentación externa en los rieles laterales.

III. PREGUNTAS DE REPASO

III-A ¿Qué pines del microcontrolador reciben la alimentación externa para no depender del puerto USB?

Para energizar externamente la Raspberry Pi Pico W de forma segura sin depender del bus USB, se utiliza el pin **VSYS** (pin físico número 39). Este pin admite una tensión de entrada flexible regulada que oscila entre 3.3V y 5.5V, la cual es procesada por el convertidor buck-boost interno de la tarjeta para establecer los 3.3V requeridos por el chip RP2040. La referencia negativa o común de la fuente externa debe conectarse a cualquiera de los pines de tierra **GND** de la placa (por ejemplo, el pin físico número 38 o 40).

III-B Explica el funcionamiento de una configuración pull-down en un botón

Una configuración *Pull-Down* fija un estado lógico bajo (0 lógico, equivalente a 0 V) en el pin del microcontrolador cuando el pulsador está en reposo (abierto). Esto se logra conectando el pin de lectura digital hacia tierra (*GND*) a través de una resistencia de valor alto (típicamente 10 kΩ). Al presionarse el botón (cerrado), se establece un camino de baja impedancia directo a la línea de voltaje positivo (3.3 V), superando la influencia de la resistencia e inyectando un estado lógico alto (1 lógico) en el microcontrolador. Esta topología previene condiciones de alta impedancia o “pines flotantes” que capten ruido electromagnético.

III-C ¿Qué trabajo cumple la función `gpio_init()` en el código C?

La función `gpio_init(uint gpio)` provista por el SDK de la Raspberry Pi Pico inicializa los registros de control asociados a una línea GPIO específica, reiniciando su hardware a un estado por defecto limpio. Al invocarse, deshabilita las funciones alternativas periféricas del pin (como canales PWM, SPI o UART), habilita los buffers digitales de entrada/salida y deja el pin listo para ser configurado como una línea digital de propósito general de lectura o escritura.

III-D ¿Qué trabajo cumple la función `gpio_get()` en el código C?

La función `gpio_get(uint gpio)` es una función de lectura digital de bajo nivel que devuelve de forma inmediata el estado lógico real presente en las terminales eléctricas del pin GPIO especificado. Retorna un valor booleano: `true` (1 lógico) si el voltaje presente en la terminal supera el umbral de reconocimiento de alta tensión del microcontrolador (aproximadamente > 2.0 V), o `false` (0 lógico) si el nivel de tensión está cercano a cero voltios.

III-E ¿Qué trabajo cumple la función `gpio_put()` en el código C?

La función `gpio_put(uint gpio, bool value)` se encarga de fijar o escribir un estado lógico digital determinado en un pin previamente configurado como salida digital (`GPIO_OUT`). Si el parámetro `value` se declara en 1 (`true`), el microcontrolador conmuta los transistores internos para reflejar el voltaje de alimentación de 3.3 V en la salida. Si el parámetro es 0 (`false`),

drena la línea conectándola internamente a tierra para reflejar 0 V.

IV. CONCLUSIONES

- Se dominó de forma efectiva la interacción bidireccional de los pines GPIO mediante el SDK en C, comprobando que las funciones integradas en `pico_stdlib` proveen una abstracción robusta para interactuar con periféricos estándar como pulsadores y diodos emisores de luz.
- La implementación del filtrado de rebotes por software (*debouncing*) demostró ser una técnica indispensable en la ingeniería de sistemas embebidos, ya que previene la ejecución de falsas instrucciones causadas por imperfecciones físicas en los contactos mecánicos de conmutación.
- Se consiguió aislar de manera exitosa la placa Raspberry Pi Pico W del bus de alimentación USB de la computadora, permitiendo un funcionamiento completamente autónomo mediante una fuente externa conectada al pin `VSYS`, lo cual constituye una base fundamental para aplicaciones reales de robótica e IoT inalámbrico.
- La correspondencia estricta entre el nombre del archivo fuente en lenguaje C y las referencias en el script CMake garantiza el despliegue óptimo y reproducible de los binarios del sistema embebido.

ANEXOS

A. Código fuente – Pulsador.c

ANEXO A – Pulsador.c (completo)

```
#include "pico/stdlib.h"

int main() {
    const uint LED_PIN = 3;    // Pin donde
                               // se conecta el LED
    const uint BOTON = 4;      // Pin donde
                               // se conecta el boton

    gpio_init(LED_PIN);
    gpio_init(BOTON);
    gpio_set_dir(LED_PIN, GPIO_OUT); // LED
                                     // como salida
    gpio_set_dir(BOTON, GPIO_IN);    //
                                     // Boton como entrada

    while(true) {
        if (gpio_get(BOTON)) {      //
            Boton presionado?
        }
    }
}
```

```

        sleep_ms(50);           //
        Espera debounce
    if (gpio_get(BOTON)) {      //
        Confirma estado
        gpio_put(LED_PIN, 1); //
        Enciende LED
    }
} else {
    gpio_put(LED_PIN, 0);      //
    Apaga LED
}
sleep_ms(10); // Delay anti-
sobrecarga
}
}

```

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
.vscode		27/02/2026 11:29 a. m.	Carpeta de archivos	
build		02/03/2026 11:19 a. m.	Carpeta de archivos	
CMakeLists.txt		27/02/2026 11:37 a. m.	Documento de texto	1 KB
pico_sdk_import.cmake		14/06/2023 04:31 a. m.	Archivo de origen C...	4 KB
Pulsador.c		27/02/2026 11:38 a. m.	Archivo de origen C	1 KB

Figura 3: Carpeta cargada con el CMake y el código.

B. Archivo CMakeLists.txt

```

ANEXO B – CMakeLists.txt (completo)

cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_6)

pico_sdk_init()

add_executable(Pulsador
    Pulsador.c
)

target_link_libraries(Pulsador
    pico_stdlib
)

pico_add_extra_outputs(Pulsador)

```

C. Diagrama Esquemático y carpeta de la practica

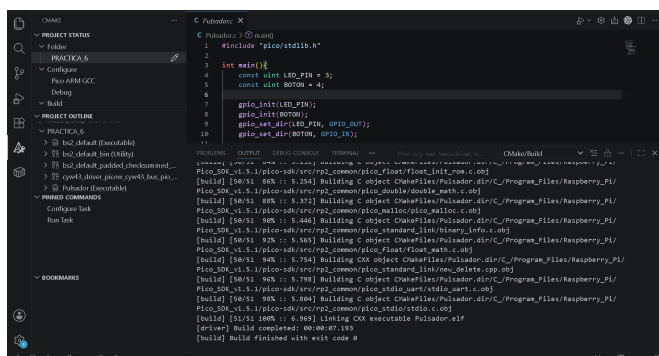


Figura 2: Código cargado exitosamente.